

Percepta en un servidor

Paso a paso para dejar de levantarlo en tu computadora

Guía de despliegue con Docker sobre un VPS Linux, escrita para **este** repositorio: los servicios que tiene hoy, los puertos que usa, los archivos que no están en git y los tres Dockerfile que todavía hay que crear.

Proyecto	Percepta — C:\dev\Camaras+IA
Destino	VPS Linux (Ubuntu Server 24.04 LTS) con Docker
Tiempo	Entre 2 y 4 horas la primera vez
Fecha	1 de septiembre de 2026

Lo que hay que entender antes de tocar nada

Hoy Percepta arranca con dos comandos: `pnpm infra:up` levanta cuatro contenedores (PostgreSQL+TimescaleDB, Redis, RabbitMQ, MinIO) y `pnpm start` arranca **seis procesos nativos** en tu Windows: identity, device, event, analytics, media-service, ai-worker y el dashboard de Angular.

Eso significa que **el docker-compose.yml del repositorio solo tiene la infraestructura**. Las aplicaciones no están contenerizadas: no existe ningún Dockerfile todavía. Por eso "subirlo con Docker" son en realidad dos trabajos, y esta guía hace los dos:

- **Contenerizar las aplicaciones** — tres Dockerfile nuevos (Node, Python y el dashboard con nginx) más un compose de producción. Están escritos completos en la Parte 4 y 5, listos para copiar y pegar.
- **Montar el servidor** — elegir el VPS, prepararlo, subir el código y los modelos, levantar todo, ponerle dominio y HTTPS.

Por qué se te traba la máquina (y qué parte hay que mover)

El que consume es el **ai-worker**: corre YOLO sobre cada frame de cada cámara, en CPU. A 6 fps con una sola cámara ya usa varios núcleos al 100% de forma sostenida. El media-service acompaña decodificando video. El resto — las cuatro APIs de Node, Postgres, Redis — casi no pesa.

O sea: mover el sistema al servidor te devuelve la máquina, pero sólo si mudás también el ai-worker. Y ahí aparece la única pregunta seria de todo el despliegue.

La cámara USB no viaja al servidor

Un VPS no tiene webcam. Hoy vos probás con la Logitech conectada por USB (source: 0), y eso **no va a funcionar** en un servidor remoto. Para probar en el VPS necesitás una fuente que se pueda alcanzar por red: una cámara IP con RTSP, o un stream de prueba. La Parte 9 explica las tres formas de resolverlo.

Las dos formas de repartir el sistema

	Opción A — todo en el VPS	Opción B — híbrido
Qué corre en el servidor	Los nueve contenedores: infraestructura, las cuatro APIs, media-service, ai-worker y el dashboard.	Infraestructura, las cuatro APIs y el dashboard. Nada de video.
Qué corre en tu red	Nada. Solo las cámaras, alcanzables desde el VPS.	media-service y ai-worker, en una PC o mini-PC al lado de las cámaras.
Te libera la máquina	Sí, completamente.	Sí, si el que captura es otra máquina y no la tuya.
Requisito	Las cámaras tienen que ser visibles desde internet o por VPN.	Esa máquina local tiene que estar siempre prendida.
Costo de video en red	Sube todo el video al VPS: ~2-4 Mbps por cámara, 24/7.	El video no sale de tu red. Solo viajan eventos y recortes.

Esta guía arma la Opción A, que es la que resuelve tu problema hoy y la más simple de operar. La Parte 13 explica cómo pasar a la B más adelante, cuando tengas varias cámaras reales y el ancho de banda empiece a importar: es el mismo compose, partido en dos máquinas.

Elegir el VPS

Cuánta máquina necesitás de verdad

La regla práctica que sale del propio sistema: el ai-worker corre a PIPELINE_FPS=6 por cámara y cada cámara con un modelo YOLO encima se come aproximadamente **un núcleo moderno completo**. Sumale 2 núcleos para Postgres, las APIs, el media-service y el sistema operativo.

Para	vCPU	RAM	Disco	Comentario
Probar, 1-2 cámaras	4 dedicados	8 GB	80 GB SSD	El mínimo con el que no vas a pelear.
Piloto real, 3-4 cámaras	8 dedicados	16 GB	160 GB SSD	Lo que recomiendo apuntar.
6+ cámaras	16 dedicados	32 GB	320 GB SSD	O pasate a Opción B, o a GPU.

Pedí CPU dedicada, no compartida

Los planes baratos venden vCPU *compartida* o *burstable*: te dejan usar el 100% unos minutos y después te estrangulan al 10-20%. La inferencia de video es exactamente la carga que dispara ese límite. Buscá en la descripción del plan las palabras **dedicated** o **dedicated vCPU**. Un plan compartido de 8 vCPU va a rendir peor que uno dedicado de 4.

Proveedores que sirven para esto

- **Hetzner Cloud** (línea CCX, CPU dedicada AMD) — la mejor relación precio/CPU. CCX23: 4 vCPU dedicadas / 16 GB / 160 GB. Datacenters en Alemania, Finlandia y Virginia (EE.UU.).
- **Contabo** — mucho más barato por núcleo, rendimiento más irregular. Sirve para probar.
- **DigitalOcean** (droplets *CPU-Optimized*) o **Vultr** (*High Frequency*) — más caros, más prolijos, con datacenter en San Pablo si te importa la latencia desde Argentina.
- **AWS / Azure / GCP** — solo si ya tenés crédito o la empresa lo exige. Para una prueba, es pagar de más y complicarse.

Los precios se mueven, así que no los pongo como si fueran fijos: a la fecha de esta guía, un servidor de 4 vCPU dedicadas con 16 GB ronda los USD 25-35 por mes en Hetzner y USD 60-90 en DigitalOcean. Verificalos antes de contratar.

Al contratar, elegí

- **Sistema operativo:** Ubuntu Server 24.04 LTS. Nada de imágenes con panel de control (cPanel, Plesk): ensucian los puertos y no aportan nada acá.
- **Ubicación:** lo más cerca posible de las cámaras, no de vos. El video sube desde las cámaras al servidor y ese es el tráfico que pesa.
- **Clave SSH:** pegá tu clave pública al crear el servidor en vez de usar contraseña. Si todavía no tenés una, en tu Windows: `ssh-keygen -t ed25519` y copia el contenido de `C:\Users\octav\.ssh\id_ed25519.pub`.
- **Backups del proveedor:** activalos. Cuestan un 20% extra y te salvan de un error propio.

PARTE 2

Preparar el servidor

Todo lo de esta parte se hace una sola vez. Conectate desde PowerShell en tu máquina (reemplaza IP_DEL_VPS por la que te dio el proveedor):

```
ssh root@IP_DEL_VPS
```

2.1 Un usuario que no sea root

en el servidor, como root

```
# Crear el usuario y darle sudo
adduser percepta
usermod -aG sudo percepta

# Copiarle tu clave SSH para poder entrar directo
rsync --archive --chown=percepta:percepta ~/.ssh /home/percepta/

# Salir y volver a entrar como percepta
exit
```

desde tu máquina

```
ssh percepta@IP_DEL_VPS
```

2.2 Actualizar y cerrar el servidor

en el servidor

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ufw fail2ban git curl rsync

# Firewall: solo SSH, HTTP y HTTPS quedan abiertos al mundo
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
sudo ufw status
```

Esto es importante y es fácil de arruinar después

Los puertos internos de Percepta — 5433 (Postgres), 6379 (Redis), 15672 (RabbitMQ), 9001 (consola de MinIO), 3001/3003/3004/3005/3010/3020 (los servicios) — **no se abren nunca al internet**. Se hablan entre contenedores por la red interna de Docker. Lo único público va a ser el dashboard, por 80 y 443. Si en algún momento estas depurando y "abris un puerto para probar", acordate de cerrarlo: la consola de MinIO y el panel de RabbitMQ tienen contraseñas de desarrollo.

2.3 Instalar Docker

en el servidor

```
# Repositorio oficial de Docker (el de Ubuntu esta viejo)
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

# Poder usar docker sin sudo
sudo usermod -aG docker $USER
newgrp docker

docker --version && docker compose version
```

2.4 Swap (si tenés 8 GB de RAM o menos)

La compilación del dashboard de Angular y la instalación de PyTorch son picos de memoria. Con 8 GB, sin swap, el build se muere sin explicación.

en el servidor

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h
```

Subir el código y los modelos

Hay dos cosas distintas que subir, y se suben distinto: **el código**, que está en git, y **los archivos pesados y los secretos**, que a propósito no están en git.

3.1 El código

Opción recomendada: un repositorio privado

en el servidor

```
# En el servidor: generar una clave para que pueda clonar
ssh-keygen -t ed25519 -C "vps-percepta" -f ~/.ssh/id_ed25519 -N ""
cat ~/.ssh/id_ed25519.pub
```

Pégala esa clave pública en tu repositorio de GitHub, en **Settings** → **Deploy keys** → **Add deploy key** (sin marcar acceso de escritura). Después, en el servidor:

en el servidor

```
cd ~
git clone git@github.com:TU_USUARIO/TU_REPO.git percepta
cd percepta
```

Opción sin repositorio: copiar directo desde Windows

desde tu máquina

```
# Desde PowerShell, parado en C:\dev
scp -r "C:\dev\Camaras+IA" percepta@IP_DEL_VPS:~/percepta
```

Es más lento y te vas a llevar `node_modules`, `.env` y `.git` al servidor sin necesidad. Sirve para la primera prueba; para trabajar de verdad, usa git: actualizar después es `git pull` en vez de recopiar todo.

3.2 Lo que git NO tiene y hay que copiar a mano

Esto es lo que más veces hace fallar un primer despliegue. El `.gitignore` del repositorio excluye a propósito:

Qué	Dónde está hoy	Por qué no está en git
<code>.env</code>	raíz del repo	Tiene contraseñas. Vas a crear uno nuevo, distinto, en el servidor.
<code>yolo8n.pt</code> , <code>yolo8n-pose.pt</code> , <code>yolo8n-seg.pt</code> , <code>yolo8s.pt</code>	raíz del repo	Pesos de modelo (6 a 22 MB cada uno). Se podrían bajar solos, pero copiarlos es más rápido y seguro.
<code>training/models/</code>	<code>epp.pt</code> , <code>epp.json</code> , <code>fall_classifier.*</code>	Artefactos de entrenamiento. El <code>epp.pt</code> son 5 horas de CPU: no lo re-entrenes en el VPS , copialo.
<code>modules/*/__pycache__</code>	—	No hace falta copiarlo, se regenera.

Copía los modelos desde PowerShell:

desde tu máquina (PowerShell; el acento invertido es el salto de línea)

```
# Los pesos base de YOLO, a la raíz del proyecto en el servidor
scp "C:\dev\Camaras+IA\yolov8n.pt" \
    "C:\dev\Camaras+IA\yolov8n-pose.pt" \
    "C:\dev\Camaras+IA\yolov8n-seg.pt" \
    "C:\dev\Camaras+IA\yolov8s.pt" \
    percepta@IP_DEL_VPS:~/percepta/

# Los modelos entrenados (EPP y caídas)
ssh percepta@IP_DEL_VPS "mkdir -p ~/percepta/training/models"
scp "C:\dev\Camaras+IA\training\models\epp.pt" \
    "C:\dev\Camaras+IA\training\models\epp.json" \
    "C:\dev\Camaras+IA\training\models\fall_classifier.pt" \
    "C:\dev\Camaras+IA\training\models\fall_classifier.onnx" \
    "C:\dev\Camaras+IA\training\models\fall_classifier.onnx.data" \
    "C:\dev\Camaras+IA\training\models\metadata.json" \
    percepta@IP_DEL_VPS:~/percepta/training/models/
```

Verificá que llegaron

En el servidor, `ls -lh ~/percepta/*.pt ~/percepta/training/models/`. Si el ai-worker arranca sin los modelos, no falla ruidosamente: levanta, no detecta nada, y te vas a volver loco buscando el error en otro lado.

Los tres Dockerfile que faltan

Estos archivos **todavía no existen** en el repositorio. Creá los cuatro en la raíz del proyecto (podés crearlos en tu Windows y subirlos con git, que es lo mejor, o pegarlos directo en el servidor con nano).

4.1 .dockerignore

Primero este, o cada build va a copiar gigabytes al vacío:

```
.dockerignore
node_modules
**/node_modules
.venv
.git
.turbo
.angular
.data
runs
training/data
training/ppe/data
training/ppe/corridas*
**/__pycache__
**/dist
*.log
.pytest_cache
```

4.2 Dockerfile.node — las cuatro APIs en una sola imagen

identity, device, event y analytics comparten el mismo monorepo y las mismas dependencias. Una imagen, cuatro contenedores que se diferencian por el comando.

```
Dockerfile.node

# Dockerfile.node - identity, device, event y analytics.
FROM node:18-bookworm-slim AS build
RUN corepack enable && corepack prepare pnpm@9.6.0 --activate
WORKDIR /app

COPY . .
RUN pnpm install --frozen-lockfile
RUN pnpm --filter @percepta/contracts build
RUN pnpm --filter @percepta/identity-service \
    --filter @percepta/device-service \
    --filter @percepta/event-service \
    --filter @percepta/analytics-service build

FROM node:18-bookworm-slim
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app /app
# El comando lo pone docker-compose; este es solo el valor por omision.
CMD ["node", "apps/identity-service/dist/main.js"]
```

4.3 Dockerfile.python — media-service y ai-worker

Los dos servicios de Python comparten imagen. Es la más pesada, porque adentro van PyTorch, Ultralytics e InsightFace: contá unos **3 GB** y entre 10 y 20 minutos de build la primera vez.

Dockerfile.python

```
# Dockerfile.python - media-service y ai-worker.
FROM python:3.11-slim-bookworm
WORKDIR /app

# libglib y libgomp los necesita OpenCV; ffmpeg, los clips en H.264;
# build-essential, las ruedas de insightface y lap que compilan al instalarse.
RUN apt-get update && apt-get install -y --no-install-recommends \
    ffmpeg libglib2.0-0 libgomp1 libgl1 build-essential \
    && rm -rf /var/lib/apt/lists/*

# PyTorch en su version de CPU: 800 MB en vez de los 2.5 GB de la de CUDA.
RUN pip install --no-cache-dir \
    torch==2.13.0 torchvision==0.28.0 \
    --index-url https://download.pytorch.org/whl/cpu

COPY packages/py-contracts ./packages/py-contracts
COPY apps/ai-worker ./apps/ai-worker
COPY apps/media-service ./apps/media-service

RUN pip install --no-cache-dir -e packages/py-contracts \
    && pip install --no-cache-dir -e apps/ai-worker \
    && pip install --no-cache-dir -e apps/media-service \
    && pip install --no-cache-dir \
        "ultralytics==8.4.115" "insightface==1.0.1" "onnxruntime==1.28.0" \
        "scikit-image>=0.26" "scipy>=1.17"

COPY modules ./modules
COPY cameras.json assignments.json ./

# Los pesos NO van en la imagen: se montan como volumen (ver compose).
CMD ["python", "-m", "media_service.main"]
```

Por que los modelos van montados y no copiados

Los .pt pesan y cambian por su cuenta: si los metés en la imagen, cada vez que re-entrenás el EPP tenés que reconstruir 3 GB. Montados como volumen, reemplazás el archivo y reiniciás el contenedor. Además, el código los busca por ruta relativa (yolo8n.pt, training/models/epp.pt), así que el punto de montaje tiene que ser exactamente ese.

4.4 Dockerfile.web — el dashboard servido por nginx

En tu máquina el dashboard corre con `ng serve` y usa `proxy.conf.json` para repartir `/api`, `/identity`, `/device`, `/media`, `/ai` y `/analytics` entre los seis servicios. En producción **ese proxy desaparece**: hay que reproducirlo en nginx, con los mismos reescrituras de ruta, o el dashboard va a dar 404 en todo.

Dockerfile.web

```
# Dockerfile.web - compila Angular y lo sirve con nginx.
FROM node:18-bookworm-slim AS build
RUN corepack enable && corepack prepare pnpm@9.6.0 --activate
WORKDIR /app
COPY . .
RUN pnpm install --frozen-lockfile
RUN pnpm --filter @percepta/contracts build
RUN cd apps/web && npx ng build --configuration production

FROM nginx:1.27-alpine
COPY --from=build /app/apps/web/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

Y el nginx que reemplaza al proxy de desarrollo:

nginx.conf

```
server {
    listen 80;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;

    # Las fotos de personas viajan en el cuerpo del pedido.
    client_max_body_size 20m;

    # /api NO se reescribe: event-service espera /api/v1/...
    location /api/ {
        proxy_pass http://event-service:3004;
        include /etc/nginx/proxy_params_percepta;
    }

    # El resto SI se reescribe: la barra final del proxy_pass
    # quita el prefijo, igual que el pathRewrite del proxy.conf.json.
    location /identity/ { proxy_pass http://identity-service:3001;
                          include /etc/nginx/proxy_params_percepta; }
    location /device/    { proxy_pass http://device-service:3003;
                          include /etc/nginx/proxy_params_percepta; }
    location /analytics/ { proxy_pass http://analytics-service:3005;
                          include /etc/nginx/proxy_params_percepta; }
    location /ai/        { proxy_pass http://ai-worker:3010;
                          include /etc/nginx/proxy_params_percepta; }

    # media-service manda MJPEG continuo: sin buffering y sin timeout,
    # o la vista en vivo se corta a los 60 segundos.
    location /media/ {
        proxy_pass http://media-service:3020;
        include /etc/nginx/proxy_params_percepta;
        proxy_buffering off;
        proxy_read_timeout 24h;
    }

    # Angular es una SPA: cualquier ruta desconocida devuelve el index.
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

proxy_params_percepta

```
proxy_http_version 1.1;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
```

Ese segundo archivo se copia dentro de la imagen agregando una línea más al `Dockerfile.web`:

Dockerfile.web (agregar al final)

```
COPY proxy_params_percepta /etc/nginx/proxy_params_percepta
```

El compose de producción

No se toca el `docker-compose.yml` que ya existe: se le suma un segundo archivo encima. Docker los combina y quedan los nueve servicios en la misma red interna.

`docker-compose.prod.yml` (primera parte)

```
# docker-compose.prod.yml - las aplicaciones, sobre la infra que ya existe.
# Uso: docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d

services:
  identity-service:
    build: { context: ., dockerfile: Dockerfile.node }
    image: percepta-node:latest
    command: ["node", "apps/identity-service/dist/main.js"]
    env_file: .env
    depends_on:
      postgres: { condition: service_healthy }
    restart: unless-stopped

  device-service:
    image: percepta-node:latest
    command: ["node", "apps/device-service/dist/main.js"]
    env_file: .env
    depends_on: [identity-service]
    restart: unless-stopped

  event-service:
    image: percepta-node:latest
    command: ["node", "apps/event-service/dist/main.js"]
    env_file: .env
    environment:
      PORT: "3004"
    depends_on: [identity-service]
    restart: unless-stopped

  analytics-service:
    image: percepta-node:latest
    command: ["node", "apps/analytics-service/dist/main.js"]
    env_file: .env
    depends_on: [identity-service]
    restart: unless-stopped
```

Y la segunda mitad del mismo archivo, sin cortar: va a continuación de analytics-service, con la misma indentación.

docker-compose.prod.yml (segunda parte)

```
media-service:
  build: { context: ., dockerfile: Dockerfile.python }
  image: percepta-python:latest
  command: ["python", "-m", "media_service.main"]
  env_file: .env
  environment:
    PORT: "3020"
    DEVICE_SERVICE_URL: "http://device-service:3003"
    EVIDENCE_DIR: "/app/.data/evidence"
  volumes:
    - evidencias:/app/.data/evidence
  depends_on: [device-service]
  restart: unless-stopped

ai-worker:
  image: percepta-python:latest
  command: ["python", "-m", "ai_worker.main"]
  env_file: .env
  environment:
    PORT: "3010"
    AI_MODULES_PATH: "./modules"
    MEDIA_SERVICE_URL: "http://media-service:3020"
    EVENT_SERVICE_URL: "http://event-service:3004"
    ANALYTICS_SERVICE_URL: "http://analytics-service:3005"
    DEVICE_SERVICE_URL: "http://device-service:3003"
    PIPELINE_FPS: "6"
  volumes:
    # Los pesos, en la ruta exacta donde el código los busca.
    - ./yolov8n.pt:/app/yolov8n.pt:ro
    - ./yolov8n-pose.pt:/app/yolov8n-pose.pt:ro
    - ./yolov8n-seg.pt:/app/yolov8n-seg.pt:ro
    - ./yolov8s.pt:/app/yolov8s.pt:ro
    - ./training/models:/app/training/models:ro
    # InsightFace baja su modelo de rostros la primera vez: que no
    # lo vuelva a bajar en cada reinicio.
    - insightface:/root/.insightface
  depends_on: [media-service, event-service]
  restart: unless-stopped

web:
  build: { context: ., dockerfile: Dockerfile.web }
  image: percepta-web:latest
  ports:
    - "127.0.0.1:8081:80"
  depends_on: [event-service, media-service, ai-worker]
  restart: unless-stopped

volumes:
  evidencias:
  insightface:
```

Fijate en el 127.0.0.1 del puerto del dashboard

"127.0.0.1:8081:80" publica el dashboard **solo hacia adentro del servidor**, no hacia internet. El que va a atender al mundo en los puertos 80 y 443 es Caddy, con HTTPS (Parte 8). Ningún otro servicio publica puertos: se hablan por la red interna de Docker usando sus nombres.

PARTE 6

El archivo .env del servidor

El .env que tenés en Windows apunta todo a 127.0.0.1 con puertos publicados. Dentro de Docker eso no existe: cada servicio se llama por su nombre y usa su puerto interno (Postgres vuelve a ser **5432**, no 5433). Además hay que cambiar las contraseñas.

en el servidor

```
cd ~/percepta
cp .env.example .env
nano .env
```

Líneas que hay que cambiar, una por una

.env (en el servidor)

```
NODE_ENV=production

# --- Base de datos: el host ahora es el nombre del contenedor ---
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=percepta
POSTGRES_USER=percepta
POSTGRES_PASSWORD=<<PONE_UNA_LARGA_Y_UNICA>>

DATABASE_URL=postgresql://percepta_app:<<OTRA_DISTINTA>>@postgres:5432/percepta
DATABASE_URL_ADMIN=postgresql://percepta:<<LA_DE_ARRIBA>>@postgres:5432/percepta

# --- Resto de la infraestructura ---
REDIS_URL=redis://redis:6379
RABBITMQ_USER=percepta
RABBITMQ_PASSWORD=<<OTRA_MAS>>
RABBITMQ_URL=amqp://percepta:<<LA_MISMA_DE_ARRIBA>>@rabbitmq:5672
RABBITMQ_MGMT_URL=http://rabbitmq:15672

S3_ENDPOINT=http://minio:9000
S3_ACCESS_KEY=percepta
S3_SECRET_KEY=<<OTRA_MAS>>
S3_BUCKET_EVIDENCES=percepta-evidences

# --- Secretos de sesion: generalos, no los inventes ---
JWT_ACCESS_SECRET=<<64_CARACTERES_AL_AZAR>>
JWT_REFRESH_SECRET=<<OTROS_64_DISTINTOS>>
# En produccion el token vuelve a durar minutos, no 8 horas.
JWT_ACCESS_TTL=900
JWT_REFRESH_TTL=1209600

# --- Puertos internos (se quedan como estan) ---
IDENTITY_SERVICE_PORT=3001
DEVICE_SERVICE_PORT=3003
EVENT_SERVICE_PORT=3004
ANALYTICS_SERVICE_PORT=3005

# --- ai-worker ---
AI_WORKER_DEVICE=cpu
AI_MODULES_PATH=./modules

# --- Retencion de evidencias: cuidan el disco ---
EVIDENCE_RETENTION_DAYS=30
EVIDENCE_PURGE_HOURS=6
```

Para generar los secretos, en el mismo servidor:

en el servidor

```
openssl rand -hex 32      # corrélo una vez por cada secreto que necesites
```

en el servidor

```
chmod 600 .env # que no lo lea nadie mas
```

El usuario admin de desarrollo sigue existiendo

La migración `0004_admin_user.sql` crea `admin@percepta.local` con contraseña `percepta`, y el dashboard inicia sesión solo con ese usuario. Está perfecto para probar, y es un agujero si el servidor queda público. Mientras sea una prueba tuya, dejalo. Antes de mostrárselo a alguien de afuera: crea el usuario real, cambiale la contraseña al admin y sacale el auto-login a `auth.service.ts`, como ya dice `EJECUTAR.md`.

Primer arranque

7.1 Construir las imágenes

en el servidor

```
cd ~/percepta
docker compose -f docker-compose.yml -f docker-compose.prod.yml build
```

La primera vez tarda entre 15 y 30 minutos: PyTorch son cientos de megas y Angular compila en serio. Las siguientes veces usa la cache y baja a un par de minutos.

7.2 Levantar la infraestructura y migrar la base

en el servidor

```
# Solo la infraestructura primero
docker compose up -d postgres redis rabbitmq minio

# Esperar a que Postgres este sano (que diga "healthy")
docker compose ps

# Aplicar el esquema completo, en el mismo orden que 'pnpm db:migrate'
docker compose exec -T postgres psql -U percepta -d percepta -v ON_ERROR_STOP=1 \
  -f /migrations/0001_init.sql \
  -f /migrations/0003_rls_hardening.sql \
  -f /migrations/0007_personas.sql \
  -f /migrations/0002_seed.sql \
  -f /migrations/0004_admin_user.sql \
  -f /migrations/0005_fall_feedback.sql \
  -f /migrations/0008_control_accesos.sql \
  -f /migrations/0009_foto_persona.sql \
  -f /migrations/0010_zona_de_trabajo.sql \
  -f /migrations/0011_kiosco.sql \
  -f /migrations/0012_plano.sql \
  -f /migrations/0013_modulo_ingreso.sql \
  -f /migrations/0014_zonas.sql \
  -f /migrations/0015_pisos.sql \
  -f /migrations/0016_epp.sql \
  -f /migrations/0017_epp_calibracion.sql
```

Ese orden no es alfabético y no es un descuido

Es exactamente el que usa el script `db:migrate` del `package.json`: el endurecimiento de RLS y la tabla de personas van **antes** del seed. Si los corrés en orden de número, el seed falla. Cuando agregues migraciones nuevas al proyecto, copia la línea del `package.json` actualizada, no la inventes.

7.3 El token de servicio

`media-service` y `ai-worker` se autentican entre ellos con un `SERVICE_TOKEN` que en tu máquina genera `tools/start-all.js` al vuelo. En Docker no hay quien lo genere, así que se crea una vez y se guarda:

en el servidor

```
# Se genera dentro del contenedor de Node, que tiene el .env y la dependencia
docker compose -f docker-compose.yml -f docker-compose.prod.yml \
  run --rm --no-deps identity-service node tools/dev-token.js service

# Copia el token que imprime y agregalo al .env:
echo 'SERVICE_TOKEN=<<EL_TOKEN_QUE_TE_DIO>>' >> .env
```

Ese token hereda el `JWT_ACCESS_TTL` del `.env`. Si lo bajaste a 900 segundos, el token de servicio vence en 15 minutos y los servicios de Python empiezan a dar 401 al rato de andar bien — un síntoma que confunde mucho. Para el token de servicio generalo con un TTL largo: corrél el comando con `-e JWT_ACCESS_TTL=31536000` agregado antes del nombre del servicio, y volvé a dejar el `.env` como estaba.

7.4 Levantar todo

en el servidor

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
docker compose -f docker-compose.yml -f docker-compose.prod.yml ps
```

Para no escribir los dos `-f` cada vez, dejalos fijos:

en el servidor

```
echo 'COMPOSE_FILE=docker-compose.yml:docker-compose.prod.yml' >> .env
# Desde ahora alcanza con: docker compose up -d
```

Dominio y HTTPS

Podés entrar por `http://IP_DEL_VPS` y ver el dashboard, pero conviene poner HTTPS **desde el principio** y no después. Dos razones concretas de este sistema:

- La pantalla de bienvenida del kiosco usa la cámara del navegador. Los navegadores **solo dan acceso a la cámara en un origen seguro**: en HTTP plano por IP, no hay forma.
- El login viaja en texto plano sobre HTTP, y con el, el token de sesión.

Apuntá un dominio (o un subdominio, sirve igual) al IP del servidor con un registro A, esperá unos minutos, y usá Caddy: saca y renueva el certificado de Let's Encrypt solo, sin configurar nada.

en el servidor

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl -sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' \
| sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' \
| sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update && sudo apt install -y caddy
```

/etc/caddy/Caddyfile

```
percepta.tudominio.com {
    encode gzip

    # El MJPEG en vivo no se puede comprimir ni bufferear.
    @vivo path /media/*
    reverse_proxy @vivo 127.0.0.1:8081 {
        flush_interval -1
    }

    reverse_proxy 127.0.0.1:8081
}
```

en el servidor

```
sudo systemctl reload caddy
sudo systemctl status caddy --no-pager
```

Listo: `https://percepta.tudominio.com` con certificado válido y renovación automática.

Qué cámaras va a ver el servidor

Ésta es la parte que no se resuelve con Docker, y la que decide si el despliegue te sirve. El media-service abre la fuente con OpenCV: acepta un número (cámara USB local) o una URL RTSP. En el VPS, el número no existe. Necesitás que la cámara sea **alcanzable por red desde el servidor**. Tres caminos:

A. Cámara IP con VPN (lo recomendado)

Instalás Tailscale en el VPS y un cliente en tu red (puede ser una Raspberry Pi, tu PC, o el propio router si lo soporta). Las cámaras quedan visibles para el servidor como si estuvieran en la misma LAN, sin abrir un solo puerto al mundo.

en el servidor

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up --accept-routes
```

Después, en el dashboard, la cámara se carga con la IP que le da Tailscale:

```
rtsp://usuario:clave@100.x.y.z:554/stream1.
```

B. Cámara IP pública con port forward

Redireccionás el puerto 554 del router a la cámara y usás tu IP pública. Anda, es lo más rápido, y es lo que **no** deberías dejar puesto: las cámaras IP baratas tienen firmware con agujeros conocidos y son de los dispositivos más escaneados de internet. Si lo usás para una prueba, poné una contraseña larga en la cámara y sacá el forward cuando termines.

C. Sin cámara: un stream de prueba

Para verificar que el sistema entero funciona antes de tener cámaras, levanta un servidor RTSP en el mismo VPS que repita un video en loop. Es la forma más rápida de ver eventos reales apareciendo en el dashboard.

en el servidor

```
# Un servidor RTSP liviano, en el mismo Docker
docker run -d --name mediamtx --network percepta_default \
  -e MTX_PROTOCOLS=tcp bluenviron/mediamtx:latest

# Publicarle un video en loop (poné cualquier .mp4 con gente caminando)
docker run -d --name fuente --network percepta_default \
  -v ~/percepta/prueba.mp4:/video.mp4:ro linuxserver/ffmpeg \
  -re -stream_loop -1 -i /video.mp4 -c copy -f rtsp rtsp://mediamtx:8554/prueba
```

Y en el dashboard cargás la cámara con origen `rtsp://mediamtx:8554/prueba`.

El nombre de la red (`percepta_default`) sale de `docker network ls`: Docker la nombra con el nombre de la carpeta del proyecto.

Cargar la cámara

Igual que en tu máquina: **Cámaras** → **Agregar cámara**, con tipo IP/WiFi y la URL RTSP. El media-service la detecta y empieza a capturar en menos de 10 segundos, sin reiniciar nada. Después arrastrás los módulos que quieras sobre ella y el ai-worker los levanta en su próximo ciclo, también sin reiniciar.

Verificar que quedó bien

Corré esto en orden. Cada paso te dice dónde está el problema si algo no responde.

Los contenedores

en el servidor

```
docker compose ps
# Los diez tienen que decir "running". minio-init y rabbitmq-init
# terminan y salen con código 0: eso es lo correcto, no un error.
```

La salud de cada servicio, desde adentro de la red

en el servidor

```
docker compose exec web sh -c '
  wget -qO- http://identity-service:3001/api/v1/health && echo
  wget -qO- http://device-service:3003/api/v1/health && echo
  wget -qO- http://event-service:3004/api/v1/health && echo
  wget -qO- http://analytics-service:3005/api/v1/health && echo
  wget -qO- http://media-service:3020/health && echo
  wget -qO- http://ai-worker:3010/health && echo
'
```

La base

en el servidor

```
docker compose exec postgres psql -U percepta -d percepta -c "\dt"
docker compose exec postgres psql -U percepta -d percepta \
  -c "SELECT count(*) FROM ai_modules;"
```

El dashboard

en el servidor

```
curl -I http://127.0.0.1:8081/ # debe dar 200
curl -I https://percepta.tudominio.com/ # debe dar 200 con HTTPS
```

Y después, a ojo, en el navegador: entrás, la sesión se inicia sola, cargás una cámara, la ves en vivo, le arrastrás un módulo y aparece un evento. Eso es el sistema completo funcionando.

Cuánto está consumiendo

en el servidor

```
docker stats --no-stream
# El ai-worker es el que manda. Si está clavado en el 100% de sus núcleos
# y la vista en vivo se entrecorta, o subís el plan o bajas PIPELINE_FPS.
```

Operarlo todos los días

Comandos que vas a usar

Para	Comando
Ver todo lo que pasa	<code>docker compose logs -f</code>
Ver un servicio solo	<code>docker compose logs -f ai-worker</code>
Las últimas 100 líneas	<code>docker compose logs --tail 100 media-service</code>
Reiniciar un servicio	<code>docker compose restart ai-worker</code>
Parar todo	<code>docker compose down</code>
Parar todo y borrar los datos	<code>docker compose down -v</code> (cuidado: borra la base)
Ver consumo en vivo	<code>docker stats</code>
Espacio en disco	<code>df -h</code> y <code>docker system df</code>

Actualizar después de un cambio

en el servidor

```
cd ~/percepta
git pull
docker compose build          # solo reconstruye lo que cambio
docker compose up -d          # reemplaza los contenedores afectados
docker compose logs -f --tail 50
```

Si el cambio incluye una migración nueva de base de datos, corré también el `psql` con el archivo nuevo, como en la Parte 7.2.

Backups

Dos cosas distintas hay que respaldar: **la base** (personas, cámaras, eventos, configuración) y **las evidencias** (los clips y snapshots). La base es chica y es la que duele perder.

~/backup-percepta.sh

```
# Un script simple, guardalo en ~/backup-percepta.sh
#!/bin/bash
set -e
DESTINO=~/.backups
mkdir -p $DESTINO
FECHA=$(date +%Y%m%d-%H%M)

cd ~/percepta
docker compose exec -T postgres pg_dump -U percepta percepta \
  | gzip > $DESTINO/percepta-$FECHA.sql.gz

# Borrar los backups de mas de 14 dias
find $DESTINO -name 'percepta-*.sql.gz' -mtime +14 -delete
echo "listo: $DESTINO/percepta-$FECHA.sql.gz"
```

en el servidor

```
chmod +x ~/backup-percepta.sh
crontab -e
# Agregar esta linea: todos los dias a las 3 de la manana
0 3 * * * /home/percepta/backup-percepta.sh >> /home/percepta/backup.log 2>&1
```

Y bajate una copia a tu máquina cada tanto, porque un backup que vive solo en el mismo servidor no es un backup:

desde tu máquina

```
scp percepta@IP_DEL_VPS:~/backups/percepta-*.sql.gz "C:\dev\backups\"
```

Restaurar

en el servidor

```
gunzip -c ~/backups/percepta-20260901-0300.sql.gz \  
| docker compose exec -T postgres psql -U percepta -d percepta
```

El disco se llena solo

Las evidencias crecen sin parar. El event-service ya tiene purga automática: se controla con EVIDENCE_RETENTION_DAYS y EVIDENCE_PURGE_HOURS en el .env. Empezá con 30 días y mirá el disco a la semana. Además, cada tanto:

en el servidor

```
docker system prune -f          # imagenes y capas viejas de builds  
docker system df                # cuanto ocupa cada cosa
```

Cuando algo no anda

Síntoma	Causa casi siempre	Qué hacer
El dashboard carga pero todo da 404 o 502	El nginx del contenedor web no tiene bien las reescrituras, o le erraste a un nombre de servicio.	Revisá que las líneas con prefijo (/identity/, /device/...) tengan barra final en el proxy_pass y /api/ no la tenga.
La vista en vivo se corta a los ~60 segundos	Falta proxy_buffering off en nginx o flush_interval -1 en Caddy.	Agregalos en las dos capas: el MJPEG es una respuesta que no termina nunca.
Los servicios de Python dan 401 después de andar bien	El SERVICE_TOKEN venció: hereda JWT_ACCESS_TTL, que en producción bajaste a 15 minutos.	Regeneralo con TTL largo (Parte 7.3) y reiniciá media-service y ai-worker.
"no se pudo abrir la fuente" en los logs de media-service	El VPS no llega a la cámara. Es lo más común de todo.	Probá desde el servidor: <code>ffprobe rtsp://...</code> . Si desde ahí tampoco entra, el problema es de red, no de Percepta.
El ai-worker levanta pero no detecta nada	Faltan los .pt: los volúmenes no montaron, o no copiaste los archivos.	<code>docker compose exec ai-worker ls -lh /app/yolov8n.pt /app/training/models/</code>
Todo va lentísimo, la vista en vivo salta	CPU al tope. Es el ai-worker.	<code>docker stats</code> . Bajá PIPELINE_FPS a 3, o quitale módulos a la cámara, o subí el plan.
El build se muere sin mensaje claro	Se quedó sin memoria compilando Angular o instalando PyTorch.	Agregá swap (Parte 2.4) y volvé a intentar.
Un servicio reinicia en loop	Casi siempre no puede conectar a Postgres.	<code>docker compose logs identity-service</code> y verificá que el .env diga @postgres:5432, no 127.0.0.1:5433.
Las migraciones fallan	Las corriste en orden numérico en vez del orden del package.json.	Empezá de cero: <code>docker compose down -v</code> y repetí la Parte 7.2 con el orden exacto.

Más adelante: partir el sistema en dos

La Opción A sube todo el video de todas las cámaras al VPS, 24 horas por día. Con una cámara de prueba no se siente; con cuatro cámaras a 2-4 Mbps cada una, ya son unos 40 GB por día y el costo de tráfico y de CPU empieza a doler.

La arquitectura que escala es partirlo: **el video se procesa donde nace** y al servidor solo suben eventos y recortes.

Dónde	Qué corre	Qué se lleva
En el VPS	postgres, redis, rabbitmq, minio, identity, device, event, analytics, web	Poca CPU. El mismo plan te aguanta muchas más cámaras.
En tu red	media-service y ai-worker, en un mini-PC junto a las cámaras	Toda la CPU de inferencia. El video no sale de la LAN.

Los cambios son chicos porque el sistema ya está hecho así: los servicios de Python se configuran enteramente por variables de entorno y hablan HTTP con los demás.

- En el VPS: sacá `media-service` y `ai-worker` del compose de producción.
- En la máquina local: un compose con esos dos servicios solos, apuntando `DEVICE_SERVICE_URL`, `EVENT_SERVICE_URL` y `ANALYTICS_SERVICE_URL` al dominio del VPS por HTTPS.
- Conectá las dos puntas con Tailscale, y que el VPS **no** exponga las APIs internas a internet: solo a la red privada.
- En el nginx del dashboard, `/media` y `/ai` pasan a apuntar al IP de Tailscale de la máquina local.

Para lo que necesitás hoy, no hagas esto todavía

La Opción A te devuelve la máquina, que es el problema que tenés ahora. Partirlo en dos agrega una segunda máquina que mantener, una VPN que puede caerse y una configuración duplicada. Es la evolución correcta cuando tengas cámaras reales y varias; no es por donde se empieza.

En resumen

El orden completo, de arriba a abajo: contratar el VPS con CPU dedicada → prepararlo (usuario, firewall, Docker, swap) → subir el código por git y los modelos por scp → crear los cuatro archivos de la Parte 4 → escribir el compose de producción y el `.env` con hosts de Docker → construir, migrar la base y generar el token de servicio → levantar → dominio y HTTPS con Caddy → conectar una cámara alcanzable → verificar de punta a punta.

Lo único que no se resuelve con esta guía es lo que dice la Parte 0: **necesitás una cámara que el servidor pueda ver**. Si todavía estas probando con la webcam USB, resolvé eso primero — con el stream de prueba de la Parte 9C alcanza para verificar el despliegue entero antes de tener la cámara real.